[Portfolio] AI-Native Web Engineering: A Journey in Lit Custom Elements

> 📝 **Developer's Note** This portfolio was developed using the **Cline + Gauss model** in practice, and its contents are structured and documented based on the real-world engineering methodology of the **Google Antigravity + Gemini 3 model**.

📌 Executive Summary

While the rise of Generative AI (LLMs) has drastically accelerated typing speed, **"Vibe Coding"** (spontaneous development relying solely on prompt-based generation without strict structure or safety nets) inevitably yields fragile, hard-to-maintain, and regression-prone codebases.

To address these challenges, by leveraging **Lit-based Web Components** as our baseline, we went beyond simple AI code generation to establish an AI-Native software development life cycle where AI assistants operate safely within rigorous, developer-defined constraints.

The project adopted a structured, step-by-step engineering methodology through five distinct phases: starting from basic Vibe Coding prototyping, establishing **AI-governed TDD (Test-Driven Development) workflows, web accessibility (a11y) rules, TS/Lit coding guidelines, decoupled scaffold architectures**, and finally automating PR validation via a **high-performance parallelized CI/CD pipeline**. Environmental issues and test timing anomalies were autonomously resolved via the automated workflow's **self-healing** capabilities.

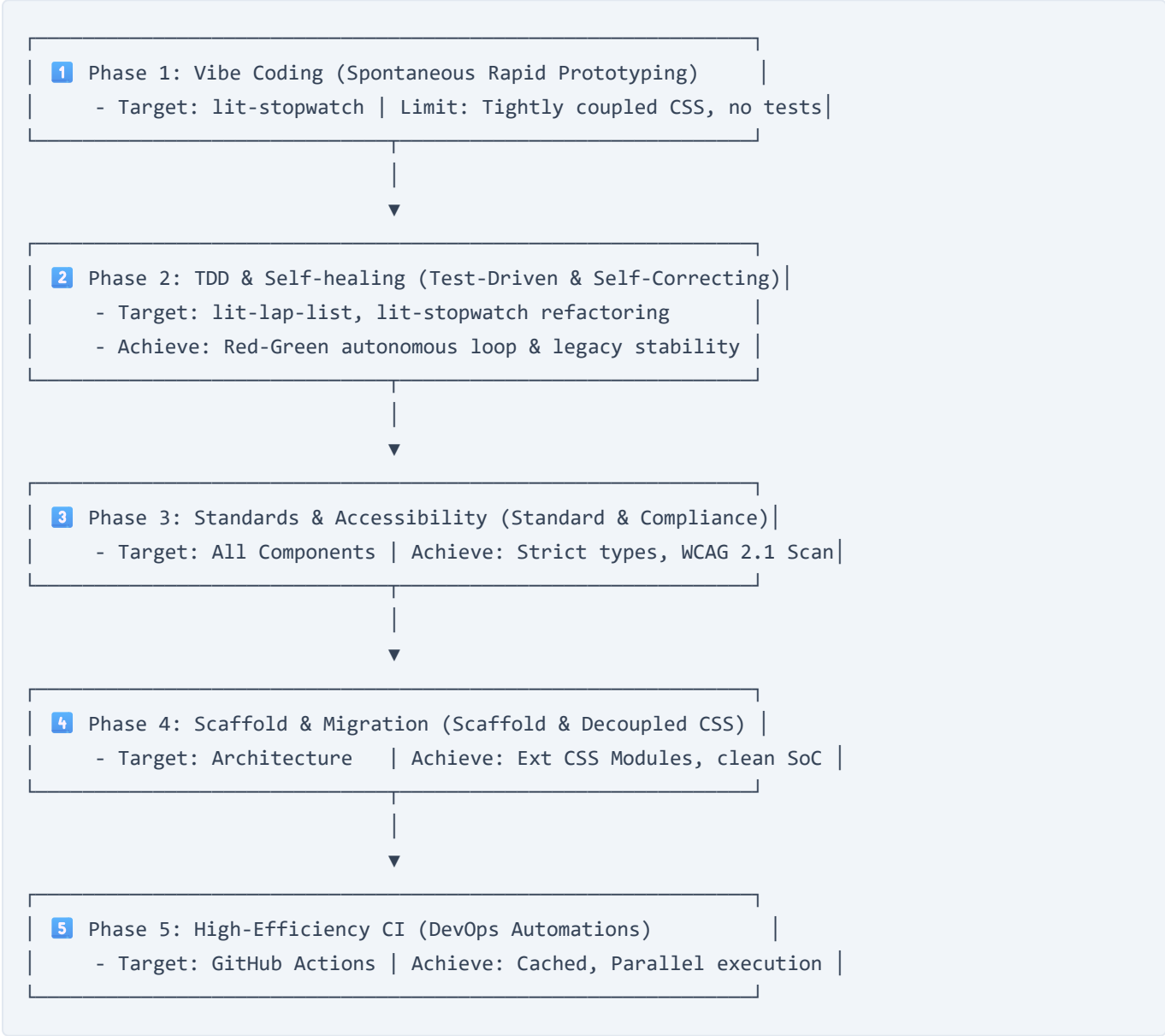
🛠 Technical Stack

Category	Technology & Tools	Objective / Role
Core Web	Lit v3, TypeScript, ES Modules	Standard Web Components UI development and encapsulation
Testing	`@web/test-runner`, `@open-wc/testing`, Mocha, Chai	Headless browser-based unit & accessibility testing
DevOps / CI	GitHub Actions, ESLint, Vite	Automated build verification, parallel linting/testing, dependency caching
AI Governance	Custom Skills, Orchestrated Red-Green-Refactor	AI development process control, self-healing loop automation

📈 Step-by-Step Engineering Methodology

To overcome the limits of Vibe Coding and guarantee production-ready software, we established a step-by-step engineering methodology consisting of five progressive phases. To ensure flawless rendering in any Markdown viewer or PDF exporter, the roadmap and summaries are represented below:

🗺️ Engineering Methodology Roadmap



📊 Methodology Summary Table

Phase	Core Paradigm	Detailed Architectural Criteria	Quantitative / Qualitative Results
Phase 1	**Vibe Coding**	Spontaneous code generation, zero initial rules	Rapid visual validation of stopwatch logic
Phase 2	**TDD & Self-healing**	Write tests first, parse trace logs, correct failures	Fully validated async state handling (lit-lap-list)
Phase 3	**Standards & A11y**	Strict TypeScript typing, WCAG automated test scan	Passed `expect().to.be.accessible()` standard
Phase 4	**Scaffold & Migration**	Component directory isolation, decoupled external CSS	Clean Separation of Concerns (SoC) using CSS module imports
Phase 5	**High-Efficiency CI**	Parallel workflow runs (Lint & Test), build caching	Cut PR verification times by over 50% with JUnit reports

---

1️⃣ Phase 1: Vibe Coding (Spontaneous Rapid Prototyping)

- **Objective**: Evaluate the raw speed of AI code generation and rapidly prototype a Lit-based stopwatch component (lit-stopwatch.ts).
- **Approach**: Prompted the AI tool to write the entire component in a single attempt without any predefined coding guidelines, style sheets, or structural rules.
- **Results & Limitations**:
  - **Pros**: A functioning interactive component was completed incredibly fast (often in just one or two prompts).
  - **Cons**: Zero unit tests left the code's reliability entirely unverified. Styling was embedded as inline strings inside the TypeScript file, making it highly fragile. Standard custom element lifecycle callbacks were poorly structured, creating potential performance and rendering anomalies.

2️⃣ Phase 2: TDD & Self-Healing Workflow (lit-lap-list)

- **Objective**: Replace the instability of Vibe Coding by establishing a strict, AI-governed **TDD (Red-Green-Refactor) workflow** to control the execution path and guarantee reliability.
- **Approach**:
  - Authored a custom workflow rule (tdd-component-workflow.md) forcing the AI to strictly proceed through: **Analysis & Design -> Write Tests First (Red) -> Run Tests & Confirm Failure -> Implement Minimal Code (Green) -> Self-Healing Loop -> Visual/Manual Verification**.
  - Applied this workflow to develop the lap times list component (lit-lap-list.ts) while simultaneously refactoring the legacy lit-stopwatch.ts from Phase 1 under these TDD guidelines to establish high unit-

test coverage and stable logic.

- **The Power of Self-Healing**:
  - During test execution on `@web/test-runner`, an asynchronous time-alignment mismatch caused a test failure (Red) due to headless browser environment latency.
  - Rather than requesting developer assistance, the automated script parsed the failed shell logs and trace logs, identified the timing discrepancy with `requestAnimationFrame`, and autonomously fixed the test by using `@open-wc/testing`'s `aTimeout` and `elementUpdated` helper primitives. All tests successfully passed (Green) without any human intervention in lit-lap-list.test.ts.

3 Phase 3: Coding Standards & Accessibility Compliance

- **Objective**: Enforce premium, industrial-grade software quality by applying strict TypeScript typing, standard Lit lifecycle patterns, **Web Accessibility (a11y)** criteria, and **TypeDoc (JSDoc)** standards.
- **Approach**:
  - Defined the custom-element-coding-rules.md standard to elevate AI coding standards.
  - **TypeScript & Class Guidelines**: Mandatory explicit types for all properties, internal states (`@state`), and method signatures. Connected and disconnected callbacks had to strictly call `super.connectedCallback()` to preserve inherited behaviors.
  - **Web Accessibility (A11y)**: Enforced semantic HTML tags, ARIA attributes (e.g., `role`, `aria-live`), logical keyboard focus flow (`tabindex="0"`, focus traps), and clear focus indicator highlights (`:focus-visible`).
  - **Auto-Documentation**: Required structured JSDoc blocks so component APIs are readable by automated generation tools.
- **Verification**:
  - Integratively ran the automated a11y scanner during the TDD loop using `@open-wc/testing`'s `accessible()` check.
  - To pass `await expect(el).to.be.accessible()`, the AI was forced to proactively design the component with accessibility markup (like `role="status"` and `aria-live="polite"`), completing the implementation without manual refactoring.

4 Phase 4: Scaffold Architecture & Structural Migration Rules

- **Objective**: Solve structural chaos as the project scales by establishing isolated folders for components and cleanly decoupling styles, tests, and source codes.
- **Approach**:
  - Standardized the component directory structure through custom-element-scaffold-rule.md.
  - **Separation of Concerns**: Extracted CSS from TypeScript files into clean, independent `.css` files.
  - **Standard-compliant CSS Modules**: Utilized standard CSS module import syntaxes (`import styles from './[name].css' with { type: 'css' }`). Integrated declarations.d.ts to allow TS compilation and introduced an automated CSS copying script (copy-css.js) to sync stylesheets with the build output.

- **Refactoring & Migration Safeguards**: Imposed a strict rule preventing the AI from modifying system-wide structures on a whim. The development flow required defining a detailed overall plan to assess impact, and only then proceeding with a structured, self-healing migration.
- Using this protocol, we safely migrated and refactored the legacy `lit-stopwatch` component into a beautifully decoupled structure inside its isolated folder: components/lit-stopwatch/.

5️⃣ Phase 5: High-Performance DevOps Parallel CI Pipeline

- **Objective**: Secure the main repository branch by validating every incoming Pull Request (PR) with an automated Build-Lint-Test pipeline, optimized to give developers near-instantaneous feedback.
- **Approach**:
  - Implemented a complete GitHub Actions CI suite (ci.yml).
  - **Parallelized Job Execution**: Once the core `build` job is verified, the pipeline triggers static code analysis (`lint`) and unit testing (`test`) **concurrently**, reducing the overall validation feedback time.
  - **State-of-the-Art Caching**: Integrated `actions/cache@v4` to cache `node_modules` against `package-lock.json` , bypassing redundant package downloads.
  - **Integrated PR Reports**: Configured `mikepenz/action-junit-report` to parse Mocha JUnit XML test results and publish an interactive visual report directly inside the GitHub PR Checks UI.

💻 Key Code Showcases

1. Decoupled CSS Modules with Modern Imports (lit-stopwatch.ts)

Demonstrating clean Separation of Concerns using native CSS Modules import syntaxes and constructable stylesheets.

```
import { LitElement, html, TemplateResult } from 'lit';
import { customElement, property, state } from 'lit/decorators.js';
// TC39 Native CSS Modules Import Standard
import styles from './lit-stopwatch.css' with { type: 'css' };

/**
 * High-fidelity, accessibility-compliant Lit stopwatch component.
 * @element lit-stopwatch
 */
@customElement('lit-stopwatch')
export class LitStopwatch extends LitElement {
  // Direct binding of isolated styles
  static styles = [styles];

  @state() private _elapsedTime: number = 0;
  @state() private _isRunning: boolean = false;
  // ... rest of logic
}
```

2. Automated Web Accessibility TDD Spec (lit-lap-list.test.ts)

Moving beyond simple unit assertions by baking WCAG 2.1 accessibility benchmarks directly into our TDD validation suite.

```
import { html } from 'lit';
import { fixture, expect } from '@open-wc/testing';
import '../lit-lap-list.js';
import { LitLapList } from '../lit-lap-list.js';

describe('LitLapList Component Accessibility', () => {
  it('should pass automated accessibility standards (WCAG 2.0/2.1)', async () => {
    const el = await fixture<LitLapList>(html`
      <lit-lap-list .laps="${[1000, 2500, 4000]}"></lit-lap-list>
    `);

    // Autonomously validates all standard contrast, markup, and screen-reader parameters
    await expect(el).to.be.accessible();
  });
});
```

3. Highly Optimized Parallel GitHub Actions Workflow (ci.yml)

Configuring a modern CI structure where build artifacts are cached and verification jobs run concurrently.

```
jobs:
  # Job 1: Compiles TS and copies CSS. Restores and warms node_modules caches.
  build:
    name: Build Verification
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Cache node_modules
        uses: actions/cache@v4
        id: node-cache
        with:
          path: node_modules
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - name: Install Dependencies
        if: steps.node-cache.outputs.cache-hit != 'true'
        run: npm ci
      - name: Compile Code
        run: npm run build

  # Job 2: Static Analysis (Runs concurrently post-build)
  lint:
    name: Lint Check
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Cache node_modules
        uses: actions/cache@v4
        with:
          path: node_modules
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - run: npm run lint

  # Job 3: Headless Browser Tests & Inline Reporting (Runs concurrently post-build)
  test:
    name: Unit Tests
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
```

```
        with:
          node-version: 20
      - name: Cache node_modules
        uses: actions/cache@v4
        with:
          path: node_modules
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - name: Run Web Test Runner
        run: npm run test
```

🤝 Retrospective

📊 Metric Analysis: Vibe Coding vs. AI-Native Engineering

Engineering Metric	Vibe Coding (Phase 1)	AI-Native Engineering (Phase 5)
Initial Velocity	**Extremely Fast** (Immediate visual layout)	**Moderate** (Requires setup of rules and CI loops)
Code Reliability	**Low** (Zero tests; updates caused anxiety)	**Very High** (Every change is verified in real-time)
Architectural Integrity	**Poor** (Embedded CSS, messy lifecycle, tight coupling)	**Excellent** (Strict folder isolation, separate stylesheets)
Onboarding & Handover Cost	**High** (Lack of API documentation and rules)	**Negligible** (Auto-JSDocs, clear scaffold rules)
DevOps & QA Verification	**Manual** (QA clicked button by button)	**100% Automated** (PR triggers parallel CI feedback)

💡 Core Takeaway for AI-Adopting Tech Organizations

- **Transition of Developer Roles**: Developers in the AI era must become orchestrators who control, manage, and supervise the system's direction to ensure the entire system is heading in the right direction.
- **The Power of Systemic Control**: Directing an LLM without any rules (Vibe Coding) easily leads to spaghetti code. However, when we establish various rules such as **TDD, Linter, Scaffolding Rule, and CI**, the AI autonomously writes and migrates (self-heals) high-quality, bug-free code at high speed within those boundaries.
- **Sustainable Software**: By maintaining modern frontend architectural standards and integrating 100% automation tools, this project explored the solid, sustainable aspects that ensure consistent quality even when new developers (or next-generation AI tools) join the project.